Atividade Prática AULA 1 – prof. Francisco Fambrini – 19/09/2026

Interfaces Web/Mobile para Coleta de Dados: Gradio x Streamlit

1. Objetivo

Desenvolver a mesma aplicação de coleta de dados duas vezes, usando duas bibliotecas Python diferentes de criação de interfaces web — Gradio e Streamlit — para comparar na prática as vantagens, limitações e fluxos de publicação de cada uma.

Ao final da atividade, o aluno terá:

- Construído uma interface web responsiva (utilizável em celular) que coleta dados de um formulário;
- Persistido os dados coletados em um arquivo .csv;
- Publicado a versão em Streamlit no Streamlit Community Cloud, obtendo um link público.

2. Tema da aplicação

A interface a ser construída é um sistema de cadastro de pacientes para um consultório médico, usado pela recepção para registrar rapidamente cada paciente que chega. O formulário deve conter os seguintes campos:

- Nome do paciente (texto)
- Idade (número)
- Convênio (seleção: Particular, Unimed, Bradesco Saúde, SulAmérica, Outro)
- Prioridade do atendimento (slider de 1 a 5, sendo 5 = urgente)
- Motivo da consulta / observações (texto)

Cada envio do formulário deve gerar uma nova linha no arquivo de dados, contendo também a data/hora do cadastro.

3. Parte 1 — Versão Gradio (execução local)

Nesta etapa a interface roda na máquina do próprio aluno e grava os dados em um .csv local.

3.1 Requisitos técnicos

- Usar gr.Blocks() (não apenas gr.Interface) para poder organizar os componentes em um layout de formulário;
- Implementar os 5 campos descritos na seção 2, cada um com o componente Gradio apropriado ao seu tipo;
- Um botão "Cadastrar" que, ao ser clicado, grava uma nova linha no arquivo pacientes.csv (criando o arquivo com cabeçalho na primeira execução, se ele não existir);
- Exibir, após salvar, uma mensagem de confirmação e a tabela com os últimos pacientes cadastrados (gr.Dataframe);
- A interface deve ser usável em tela de celular — usar demo.launch() com layout simples em coluna única (evitar gr.Row() com muitas colunas lado a lado).

3.2 Esqueleto de código (ponto de partida)

```
import gradio as gr
import pandas as pd
import os
from datetime import datetime

ARQUIVO_CSV = "pacientes.csv"
```

```
COLUNAS = ["timestamp", "nome", "idade", "convenio", "prioridade", "motivo"]

def cadastrar_paciente(nome, idade, convenio, prioridade, motivo):
    linha = {
        "timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "nome": nome, "idade": idade,
        "convenio": convenio, "prioridade": prioridade,
        "motivo": motivo,
    }
    novo = pd.DataFrame([linha])
    if os.path.exists(ARQUIVO_CSV):
        novo.to_csv(ARQUIVO_CSV, mode="a", header=False, index=False)
    else:
        novo.to_csv(ARQUIVO_CSV, mode="w", header=True, index=False)
    return "Paciente cadastrado com sucesso!", pd.read_csv(ARQUIVO_CSV).tail(5)

with gr.Blocks() as demo:
    gr.Markdown("## Cadastro de Pacientes")
    nome = gr.Textbox(label="Nome do paciente")
    idade = gr.Number(label="Idade")
    convenio = gr.Dropdown(
        ["Particular", "Unimed", "Bradesco Saúde", "SulAmérica", "Outro"],
        label="Convênio",
    )
    prioridade = gr.Slider(1, 5, step=1, label="Prioridade do atendimento")
    motivo = gr.Textbox(label="Motivo da consulta / observações", lines=3)
    botao = gr.Button("Cadastrar")
    saida_msg = gr.Textbox(label="Status", interactive=False)
    tabela = gr.Dataframe(label="Últimos pacientes cadastrados")
    botao.click(
        cadastrar_paciente,
        [nome, idade, convenio, prioridade, motivo],
        [saida_msg, tabela],
    )

demo.launch()
```

Este código é apenas um ponto de partida — o aluno pode incluir campos adicionais (ex.: telefone, data de nascimento) se desejar.

4. Parte 2 — Versão Streamlit (com deploy na nuvem)

Nesta etapa o aluno recria a mesma aplicação usando Streamlit e a publica no Streamlit Community Cloud.

4.1 Requisitos técnicos

- Mesmos campos e mesma lógica de gravação da versão Gradio, agora usando st.form() para agrupar as entradas;
- Usar st.session_state para acumular os pacientes cadastrados durante a sessão;
- Atenção: o sistema de arquivos do Streamlit Community Cloud é efêmero — o .csv gravado lá não é permanente entre reinicializações do app. O aluno deve implementar também um botão de download do CSV (st.download_button) para que a recepção possa exportar os dados cadastrados na sessão;
- Criar um repositório no GitHub contendo o app.py e um requirements.txt (com gradio, streamlit, pandas);
- Publicar o app no Streamlit Community Cloud (share.streamlit.io), conectando o repositório do GitHub.

4.2 Esqueleto de código (ponto de partida)

```
import streamlit as st
import pandas as pd
```

```
from datetime import datetime

st.set_page_config(page_title="Cadastro de Pacientes", layout="centered")
st.title("Cadastro de Pacientes")

if "pacientes" not in st.session_state:
    st.session_state.pacientes = pd.DataFrame(
        columns=["timestamp", "nome", "idade", "convenio", "prioridade", "motivo"]
    )

with st.form("cadastro", clear_on_submit=True):
    nome = st.text_input("Nome do paciente")
    idade = st.number_input("Idade", min_value=0, max_value=120, step=1)
    convenio = st.selectbox(
        "Convênio",
        ["Particular", "Unimed", "Bradesco Saúde", "SulAmérica", "Outro"],
    )
    prioridade = st.slider("Prioridade do atendimento", 1, 5)
    motivo = st.text_area("Motivo da consulta / observações")
    enviado = st.form_submit_button("Cadastrar")

if enviado:
    nova_linha = {
        "timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "nome": nome, "idade": idade,
        "convenio": convenio, "prioridade": prioridade,
        "motivo": motivo,
    }
    st.session_state.pacientes = pd.concat(
        [st.session_state.pacientes, pd.DataFrame([nova_linha])],
        ignore_index=True,
    )
    st.success("Paciente cadastrado!")

st.dataframe(st.session_state.pacientes.tail(5))

csv = st.session_state.pacientes.to_csv(index=False).encode("utf-8")
st.download_button("Baixar CSV", csv, "pacientes.csv", "text/csv")
```

4.3 Passo a passo do deploy

1. Criar um repositório público no GitHub com o app.py e o requirements.txt;
2. Acessar share.streamlit.io e fazer login com a conta do GitHub;
3. Clicar em "New app", selecionar o repositório, a branch e o arquivo app.py;
4. Aguardar o build e testar o app pelo link público gerado (formato *.streamlit.app);
5. Testar o formulário também pelo celular, acessando o mesmo link.

5. Entrega

- Link do repositório GitHub (contendo as duas versões: pasta gradio_app/ e pasta streamlit_app/);
- Link público do app publicado no Streamlit Community Cloud;
- Print (captura de tela) da interface Gradio rodando localmente;
- Arquivo pacientes.csv de exemplo gerado durante os testes.